

Trabalho Final – Programação Lógica

Jogo da Velha em Prolog

Disciplina: Programação Lógica

Alessandra Corrêa

1. Introdução

Este trabalho apresenta a implementação de um Jogo da Velha utilizando a linguagem Prolog, seguindo o paradigma da programação lógica. Em vez de utilizar estruturas imperativas tradicionais, o comportamento do jogo é descrito por fatos e regras lógicas, permitindo que o próprio mecanismo de inferência do Prolog determine as respostas e estados do jogo.

2. Descrição do Jogo

O jogo consiste em dois jogadores que se alternam marcando posições em um tabuleiro 3x3. O jogador x vence ao completar uma linha, coluna ou diagonal com três símbolos iguais. Caso todas as posições sejam preenchidas sem vencedor, ocorre empate.

3. Modelagem do Conhecimento

Cada jogada é representada pelo termo jogada(Linha, Coluna, Jogador). O estado do jogo é uma lista dessas jogadas. As regras de vitória, empate, validação e alternância de jogadores são modeladas declarativamente.

4. Regras Principais

O programa utiliza predicados como vencedor/2, empate/1 e jogar/5. As condições de vitória são verificadas por meio de cláusulas alternativas e o Prolog utiliza o backtracking automaticamente para testar todas as possibilidades.

- Consultas Básicas
- ?- jogar([], x, 1,1, E1).
- ?- jogar(E1, o, 2,2, E2).
- ?- jogar(E2, x, 1,2, E3).
- ?- jogar(E3, o, 3,3, E4).
- ?- jogar(E4, x, 1,3, E5).

Consultas de Vitória

?- vencedor(x, [jogada(1,1,x), jogada(1,2,x), jogada(1,3,x)]).

?- vencedor(J, [jogada(1,1,x), jogada(1,2,x), jogada(1,3,x)]).

Consultas de Empate

?- empate([jogada(1,1,x), jogada(1,2,o), jogada(1,3,x), jogada(2,1,x), jogada(2,2,o), jogada(2,3,o), jogada

Consultas Auxiliares

?- mostrar_tabuleiro([jogada(1,1,x), jogada(2,2,o), jogada(1,2,x)]).

?- casas_livres([jogada(1,1,x), jogada(2,2,o)], L).

?- contar_pecas(x, [jogada(1,1,x), jogada(1,2,x), jogada(2,2,o)], N).

Consultas de Teste de Erro

?- jogar([], x, 4,1, E).

?- jogar([jogada(1,1,x)], o, 1,1, E).

Consulta de Demonstração

?- demo.

Sequência Completa de Partida

Jogada 1

?- jogar([], x,1,1,E1).

Jogada 2

?- jogar([jogada(1,1,x)], o,2,2,E2).

Jogada 3

?- jogar(

[jogada(2,2,o), jogada(1,1,x)],x,1,2,E3).

E assim por diante.

Para mostrar o tabuleiro:

mostrar_tabuleiro([

jogada(1,1,x),

jogada(2,2,o),

jogada(1,2,x)

]).

Verifica o vencedor:

vencedor(x,

[jogada(1,1,x),

jogada(1,2,x),

jogada(1,3,x)]).

Verifica empate:

empate([

jogada(1,1,x),

jogada(1,2,o),

jogada(1,3,x),

jogada(2,1,x),

jogada(2,2,o),

```
jogada(2,3,o),  
jogada(3,1,o),  
jogada(3,2,x),  
jogada(3,3,x)  
]).
```

Rodar jogadas de demonstração.

demo.

6. Conclusão

O projeto demonstrou que é possível implementar um jogo completo utilizando exclusivamente o paradigma declarativo da programação lógica. O Prolog permitiu representar regras e estados de forma natural, utilizando unificação, inferência lógica, negação por falha e backtracking.